

Assignment 7: Secure Network Application Development Using TCP

Total Marks: 100

Objective

Enhance the GUI-based multi-client TCP application by implementing practical security mechanisms including authentication, password protection, session management and input validation. The emphasis is on application security rather than advanced cryptography.

Learning Outcomes

- Understand authentication and authorization.
- Implement secure password storage using SHA-256 hashing.
- Validate user input.
- Prevent duplicate logins.
- Implement session timeout and secure logging.
- Verify authentication using Wireshark.

Prerequisite

Reuse and extend Assignment 6. Do not develop a new application from scratch.

Network Setup

Use Mininet:

```
sudo mn --topo single,5
```

Verify using:

```
nodes  
net  
pingall
```

Development Guidelines

- Reuse the existing GUI and networking code.
- Keep networking logic independent from the GUI.
- Use `hashlib.sha256()` for password hashing.
- Use CSV or JSON for storing user credentials.
- Never store or display plaintext passwords.

Task 1: User Authentication (20 Marks)

Implement username and password based authentication.

Task 2: Secure Password Storage (15 Marks)

Store passwords using SHA-256 hashes instead of plaintext.

Task 3: Duplicate Login Prevention (10 Marks)

Prevent the same user from logging in simultaneously.

Task 4: Input Validation (15 Marks)

Reject invalid usernames, empty passwords, oversized messages and unsupported commands.

Task 5: Failed Login Protection (10 Marks)

Block login temporarily after five consecutive failed attempts.

Task 6: Session Management & Secure Logging (10 Marks)

Support logout, inactivity timeout and secure logging without storing passwords.

Task 7: Wireshark Verification (10 Marks)

Capture login, failed login, authenticated communication and logout using filter tcp.port == 5000.

Task 8: GitHub Update (5 Marks)

Update the repository created in Assignment 6 with the latest code, report and README.

Task 9: Handwritten Reflection (5 Marks)

Write the reflection answers by hand, scan them and submit with the report.

Implementation Hints

- Maintain a simple user database.
- Hash passwords before saving them.
- Track logged-in users in memory.
- Use timestamps to implement session timeout.
- Do not print passwords in console logs.

Required Outputs

- server.py
- client_gui.py
- users.csv or users.json
- security_log.txt
- screenshots/
- report.pdf
- handwritten_reflection.pdf (or scanned images)

Handwritten Reflection Questions

1. Explain the difference between authentication and authorization.

2. Why should passwords never be stored in plain text?
3. What is the purpose of password hashing?
4. How does duplicate login prevention improve security?
5. Suggest two additional security improvements for your application.

These answers MUST be written by hand on separate sheets, scanned clearly and submitted along with the report. Typed answers will not be accepted.

GitHub Submission (Mandatory)

- Continue using the public GitHub repository created in Assignment 6.
- Update the repository with the source code, report, screenshots and README.
Submit the GitHub repository link along with the ZIP file.

Report Format

- Objective
- Security Features Implemented
- System Architecture
- Implementation
- Testing
- Wireshark Verification
- Conclusion

Evaluation Rubric

Component	Marks
User Authentication	20
Secure Password Storage	15
Duplicate Login Prevention	10
Input Validation	15
Failed Login Protection	10
Session Management & Secure Logging	10
Wireshark Verification	10
GitHub Update	5
Handwritten Reflection	5
Total	100

Important Notes

- Reuse Assignment 6 implementation.
- Focus on practical security features.
- Generate your own screenshots.
- Be prepared to demonstrate the application.
- Submissions without handwritten reflections will be treated as incomplete.